

Generierung von Task Graphs

Anja Graf



Lehrstuhl für Embedded Systems
Institut für Informatik
Universität Augsburg

06. August 2025

Zielsetzung des Forschungsmoduls

Funktionsweise und Inhalt

- Graphgenerierung mit Erdős–Rényi Modell

- Taskparameter

- Uniprocessor Scheduling

- Utilization

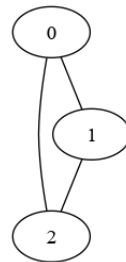
 - Unifast Algorithmus

 - Dirichlet Rescale Algorithmus

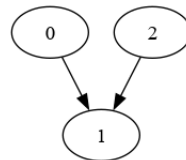
Ergebnisse und Beispiele



- ▶ Generieren eines Taskgraphen mit randomisierter Graphstruktur
 - ▶ Dabei: Knoten sind Tasks, Kanten sind Abhängigkeiten
 - ▶ Graph ist Directed und Acyclic (DAG)
- ▶ Command Line Interface für Benutzerschnittstelle
 - ▶ Übergabeparamter wie Knoten- oder Kantenanzahl
- ▶ Output in Form von generierten Dateien



Ungerichteter, zyklischer Graph
Mit 3 Knoten, 3 Kanten



Gerichteter, azyklischer Graph
Mit 3 Knoten, 2 Kanten

Zwei Varianten des Modells:



Zwei Varianten des Modells:

$G(n, p)$

n = Anzahl an Knoten

p = Wahrscheinlichkeit für Kanten

- ▶ Iteriere über geschuffelte Knotenvektoren, um Knoten i, j für jede mögliche Kante $e = \{i, j\}$ zu erhalten ($i \neq j$)
- ▶ Füge diese Kante e mit Wahrscheinlichkeit p hinzu



Zwei Varianten des Modells:

$G(n, p)$

n = Anzahl an Knoten

p = Wahrscheinlichkeit für Kanten

- ▶ Iteriere über geshuffelte Knotenvektoren, um Knoten i, j für jede mögliche Kante $e = \{i, j\}$ zu erhalten ($i \neq j$)
- ▶ Füge diese Kante e mit Wahrscheinlichkeit p hinzu

$G(n, m)$

n = Anzahl an Knoten

m = Anzahl an Kanten

- ▶ Wähle zwei zufällige Knoten aus
- ▶ Füge Kante zwischen Knoten hinzu, falls neue Kante
- ▶ Iteriere, bis Anzahl m erreicht



Zwei Varianten des Modells:

$G(n, p)$

n = Anzahl an Knoten

p = Wahrscheinlichkeit für Kanten

- ▶ Iteriere über geschuffelte Knotenvektoren, um Knoten i, j für jede mögliche Kante $e = \{i, j\}$ zu erhalten ($i \neq j$)
- ▶ Füge diese Kante e mit Wahrscheinlichkeit p hinzu

$G(n, m)$

n = Anzahl an Knoten

m = Anzahl an Kanten

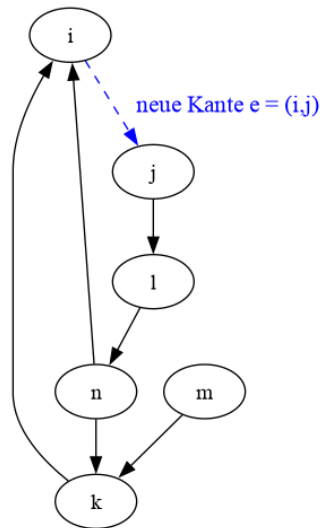
- ▶ Wähle zwei zufällige Knoten aus
- ▶ Füge Kante zwischen Knoten hinzu, falls neue Kante
- ▶ Iteriere, bis Anzahl m erreicht

Insgesamt sind beide Varianten noch ungerichtet und prüfen nicht auf Kreisfreiheit!



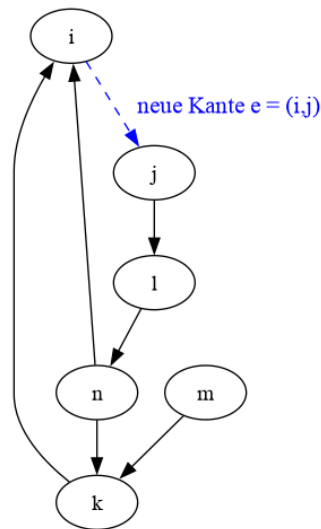
Daher:

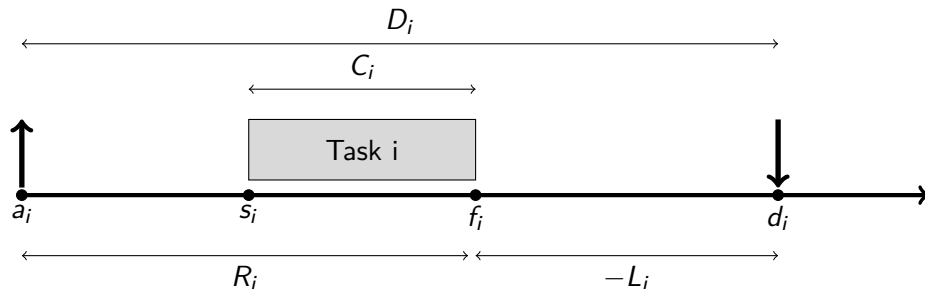
- ▶ Füge gerichtete, statt ungerichtete Kanten ein
- ▶ Prüfe beim Hinzufügen ob eine der beiden Kanten (i, j) oder (j, i) bereits existiert
- ▶ Prüfe, ob es bereits Pfad j nach i gibt mittels rekursiven Tiefensuche
- ▶ Falls ja, würde neue Kante $e = (i, j)$ Kreis schließen
 - ▶ Darf nicht eingefügt werden



Daher:

- ▶ Füge gerichtete, statt ungerichtete Kanten ein
- ▶ Prüfe beim Hinzufügen ob eine der beiden Kanten (i, j) oder (j, i) bereits existiert
- ▶ Prüfe, ob es bereits Pfad j nach i gibt mittels rekursiven Tiefensuche
- ▶ Falls ja, würde neue Kante $e = (i, j)$ Kreis schließen
 - ▶ Darf nicht eingefügt werden
- ▶ Graph ist nun Directed und Acyclic ✓





Computation time C_i

Arrival/Release time a_i

Absolute Deadline d_i

Relative Deadline $D_i = d_i - a_i$

Start time s_i

Finishing time f_i

Response time $R_i = f_i - a_i$

Lateness $L_i = f_i - d_i$

Priority, set to i



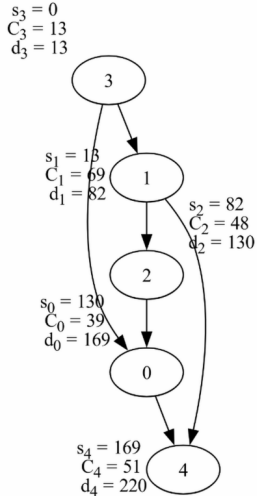
Ziel: Ausführungsreihenfolge finden

- ▶ Dabei sollen Abhängigkeiten berücksichtigt werden
- ▶ Also falls Kante $(i, j) \in E$ für Graph $G = (V, E)$ muss Task i vor Task j ausgeführt worden sein
- ▶ Dafür generieren wir zu Beginn eine Hash Map
 - ▶ Keys: Knoten bzw. Task
 - ▶ Values: Jeweils eine Liste aller Vorgänger (z.B. $j \in$ Value Liste von Key i)
- ▶ Dann fügen wir iterativ bereitstehende Tasks mit der höchsten Priorität in den Schedule ein
 - ▶ Task ist bereit/ready, falls vorher keine abhängigen Tasks mehr ausgeführt werden müssen



```
schedule  $\leftarrow$  [], ready  $\leftarrow$  {}, pre  $\leftarrow$  get_predecessor_hash_map()
forall key, value  $\in$  pre : value  $= \emptyset$  do
    | ready.add(key)
end
// throw error if ready =  $\emptyset$ 
while ready  $\neq \emptyset$  do
    | highest_priority  $\leftarrow$  ready.get_min()
    | ready.remove(highest_priority)
    | pre.remove(highest_priority)
    | schedule.append(highest_priority)
    | forall key, value  $\in$  pre : highest_priority  $\in$  value do
        | value  $\leftarrow$  value  $\setminus$  highest_priority
        | if value  $= \emptyset$  then
            | ready.add(key)
        | end
    | end
end
return schedule
```





- Beispielhafter Output des Programms bei Aufruf mit Parametern *–nodes 5 –edges 6* und Auswahl des $G(n,m)$ Erdős–Rényi Modell

Task Parameter

Computation time C_i is set to random value in $\{10..100\}$

Arrival/Release time a_i is set to 0 for all i

Absolute Deadline d_i is set to $s_i + C_i$

Relative Deadline $D_i = d_i - a_i = d_i$

Start time s_i

Finishing time f_i is set to d_i

Response time $R_i = f_i - a_i = f_i$

Lateness $L_i = f_i - d_i = 0$

Priority of task is defined by its number

Schedule: [3, 1, 2, 0, 4]



- ▶ Weitere Anforderung an die Parameterverteilung:
Berücksichtigung der Auslastung von Tasks
- ▶ Utilization für Task i definiert als $U_i = \frac{C_i}{T_i}$
 - ▶ dabei T_i Periode, d.h. fester Zeitabstand, in welchem die Task i wiederholt ausgeführt werden soll
- ▶ Gesamtauslastung des Prozessors $U = \sum_{i=1}^n U_i = \sum_{i=1}^n \frac{C_i}{T_i}$
 - ▶ „Wie viel Prozessorzeit pro Zeiteinheit benötigen alle Tasks zusammen?“
 - ▶ Wenn $U > 1$: Prozessor ist überlastet und Tasks sind nicht schedulable/echtzeitfähig
- ▶ Im Programm kann zwischen Unifast und Drs Algorithmus gewählt werden
 - ▶ Dafür wird Parameter U mitgegeben und daraus ein Vektor $U_1, U_2, \dots, U_n$ berechnet
 - ▶ Die entsprechenden Werte werden für jede Task i angepasst, damit das Verhältnis mit U_i übereinstimmt



```
fn uunifast(&self, u: f64) -> Vec<f64> {
    assert!(u <= 1.0 && u >= 0.0, "The processor utilization
        should be percentage between 0 and 1");
    let mut sum_u = u;
    let mut rng = rand::thread_rng();
    let mut vect_u: Vec<f64> = Vec::new();
    for i in 0..self.nodes.len() - 1 {
        let next_sum_u = sum_u * rng.r#gen:::<f64>().powf(1.0 /
            (self.nodes.len() - i) as f64);
        vect_u.push(sum_u - next_sum_u);
        sum_u = next_sum_u;
    }
    vect_u.push(sum_u);
    println!("{:?}", vect_u);
    assert!(u - vect_u.iter().sum:::<f64>().abs() < 1e-6);
    vect_u
}
```

In uunifast.rs, ab Zeile 34



Constraints für generierte Graphen

Gilt $\forall i \in \{0..n-1\}$, n = Anzahl der Tasks, i = Index der Task im Schedule

► $C_i \in \{10..100\}$

Die Computation time muss für alle Knoten zwischen 10 und 100 liegen.

► $d_i \geq C_i + s_i$

Die Deadline muss spät genug sein, damit die Task vorher ausgeführt werden kann.

► $s_i \geq d_{i-1}$, wobei $s_0 = 0$

Eine Task, welche früher gescheduled ist, muss beendet worden sein bevor die nächste startet. Zur Vereinfachung wird $s_i = d_{i-1}$ gesetzt.

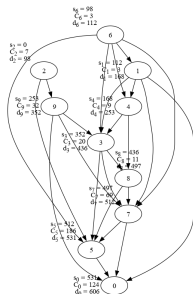
► $U_i = \frac{C_i}{T_i}$

Das Verhältnis aus Computation time und Periode muss der berechneten Utilization entsprechen. Dabei wird der Einfachheit halber $T_i = d_i$ gleichgesetzt.









Task Parameter

Computation time C_i is set to random value in $\{10, 100\}$

Arrival/Release time s_i is set here to 0 for all i

Absolute Deadline d_i is set here to $s_i + C_i$

Relative Deadline $D_i = d_i - s_i = d_i$

Start time s_i

Finishing time f_i is set here to d_i

Response time $R_i = f_i - s_i = f_i$

Lateness $L_i = f_i - d_i = 0$

Priority of task is defined by its number

Schedule: [2, 6, 1, 4, 9, 3, 8, 7, 5, 0]

